KTÜ OF TEKNOLOJI FAKÜLTESI · YAZILIM MÜHENDISLIĞI BÖLÜMÜ

Nesne Yönelimli Programlama

2026–2027 Güz Dönemi · Laboratuvar Föyü

HAFTA 07

Laboratuvar Föyü

İki seviyeli hiyerarşi · `super` ile yapıcı zinciri · ortak davranışı üst sınıfa taşıma · IS-A / HAS-A kararı · kalıtımdan bileşime

ÖZET

Bu laboratuvar oturumunda bir kütüphane yönetim sisteminin sınıf hiyerarşisini sıfırdan kuracaksınız. Önce ortak paydayı taşıyan `Yayin` üst sınıfını ve ondan türeyen `Kitap` sınıfını yazacaksınız. İkinci görevde `Dergi` ve `AkademikDergi` sınıflarıyla **üç seviyeli** bir hiyerarşi kurup `super` zincirinin nasıl işlediğini çıktı üzerinde göreceksiniz. Üçüncü görevde size verilen, aynı kodu iki kez içeren bir tasarımı ortak paydayı üst sınıfa taşıyarak düzelteceksiniz. Dördüncü görevde bir IS-A / HAS-A kararını gerekçelendirecek, bonus görevde ise bozuk bir kalıtımı bileşime dönüştürerek hiyerarşinin tamamını tek bir katalog üzerinden çalıştıracaksınız.

Ders	Nesne Yönelimli Programlama (Zorunlu)
Program / Sınıf	Yazılım Mühendisliği · 2. Sınıf · Güz
Haftalık Yük	2 saat teori + 2 saat laboratuvar (toplam 4 saat)
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE
Laboratuvar Sorumlusu	Arş. Gör. Betül MUMCU
Programlama Dili	Java (JDK 21 LTS)
Geliştirme Ortamı	Visual Studio Code + Extension Pack for Java

Oturum Bilgileri

	Bilgi
Süre	120 dakika (2 ders saati)
Sorumlu	Arş. Gör. Betül MUMCU
Ortam	Visual Studio Code + JDK 21 (LTS)
Çalışma biçimi	Bireysel
Ön koşul	Hafta 7 ders notu Bölüm 3 ve 4 okunmuş olmalıdır
Teslim	Tüm `.java` dosyaları tek bir sıkıştırılmış klasör hâlinde, oturum bitiminde
Dosya adı biçimi	`Hafta07_<Numara>_<AdSoyad>.zip`
Değerlendirme	Görev 1–4 zorunlu (80 puan) · Görev 5 ileri düzey (20 puan)

Bu Bölümün Sonunda Öğrenci

- Bir üst sınıf tasarlayıp ondan `extends` ile alt sınıf türetir; ortak paydayı doğru yerde toplar.
- `super(...)` ile yapıcı zincirini kurar; üst sınıfın doğrulamalarını alt sınıfta tekrarlamaz.
- Bir metodu `@Override` ile ezer ve `super.metot()` ile üst sürümü yeniden kullanır; üst sınıfın kurallarını korur.
- `protected` erişim düzeyini gerekçeli biçimde kullanır; gereksiz yerde `protected` yazmaz.
- Üç seviyeli bir hiyerarşide kuralların hangi sırayla işlediğini çalışan çıktı üzerinde açıklar.
- Kod tekrarını ortak paydayı üst sınıfa taşıyarak (pull up) giderir; kazancı satır sayısıyla ölçer.
- “X bir Y'dir” cümle sınamasıyla IS-A / HAS-A kararı verir ve bozuk bir kalıtımı bileşime dönüştürür.



Şekil 7.1 — Laboratuvar oturumunun zaman planı

Çalışma Kuralları

1. Her `public` sınıf için **ayrı bir `.java` dosyası** oluşturun. Dosya adı, içindeki `public` sınıfın adıyla birebir aynı olmalıdır.
2. Kaynak dosyaları `src`, derlenmiş dosyaları `bin` klasöründe tutun: `javac -d bin src/*.java`

3. Tanımlayıcılarda (sınıf, metot, değişken adları) **Türkçe karakter kullanmayın**. Ekrana yazdırılan metinlerde de ASCII kalın.
4. Ezdiğiniz **her** metodun üstüne `@Override` yazın. Bu bir stil önerisi değil, puanlama ölçütüdür.
5. Üst sınıf davranışını genişletirken `super.metot()` çağrısını bilinçli olarak verin ya da vermeyin; kararınızı bir yorum satırıyla gerekçelendirin.
6. Alanları `private` tutun; alt sınıfın **doğrudan** erişmesi gerekiyorsa `protected` yapın ve nedenini yorum satırında yazın.
7. Alt sınıfta üst sınıftaki bir alan adını **tekrar kullanmayın** (alan gölgeleme; ders notu Bölüm 6.1).
8. Görevleri sırayla yapın; her görev bir öncekinin sınıflarını kullanır.

> **İNTİHAL UYARISI**
>
> Teslim edilen kod **bireysel** çalışmanın ürünü olmalıdır. Birbirinin kopyası olduğu belirlenen teslimlerin tamamı değerlendirme dışı bırakılır. Arkadaşınızla tartışabilirsiniz; kod alışverişi yapamazsınız.

Görev 0 — Ortam ve Ön Koşul Doğrulaması

Süre	Puan	Zorluk
10 dakika	Puansız — **zorunlu**	●○○

Amaç

Java ortamının çalıştığını doğrulamak ve haftanın üç yapıtaşını (`extends`, `super`, `@Override`) en küçük ölçekte çalıştırmak.

Görev Tanımı

`Gorev0_Kontrol` adında bir dosya oluşturun. İçinde `OnKosulUst` ve ondan türeyen `OnKosulAlt` sınıfları bulunsun. `main` metodu Java sürümünü yazdırsın, ardından mirasın çalıştığını üç ayrı sınamayla göstersin.

Gereksinimler

1. `OnKosulUst` sınıfının `protected final String etiket` alanı ve bir parametreli yapıcısı olmalıdır.
2. `OnKosulAlt` sınıfı `extends OnKosulUst` yazmalı ve yapıcısında `super(etiket)` çağrısını yapmalıdır.
3. `OnKosulAlt` sınıfı `selam()` metodunu `@Override` ile ezmeli ve gövdesinde `super.selam()` çağrısını kullanmalıdır.
4. `main` metodu `instanceof` ile iki tür sınaması yapmalıdır: alt tür ve üst tür için.
5. `main` metodu `getSuperclass()` çağrılarıyla `Object` köküne kadar olan zinciri yazdırmalıdır.

Beklenen Çıktı

```
▸ TERMINAL — java Gorev0_Kontrol
=== Hafta 07 Laboratuvari - Ortam Kontrolu ===
Java surumu : 25.0.2
Isletim sis.: Linux

1) extends ve super calisiyor:
   UST sinif: Hafta07 + ALT sinif eki
2) IS-A iliskisi:
   alt instanceof OnKosulAlt -> true
   alt instanceof OnKosulUst -> true
3) Object koku:
   ust sinif zinciri -> class OnKosulUst -> class java.lang.Object

Uc satirin tamami goruntulendiyse ortam HAZIRDIR.
```

> **İPUCU**
>
> Java sürümünü `System.getProperty("java.version")` ile okuyabilirsiniz; sürüm numaranız farklı olabilir, bu bir sorun değildir. Bir sınıfın üst sınıfına `SinifAdi.class.getSuperclass()` ile ulaşabilirsiniz.

Kontrol Soruları

1. `OnKosulAlt` yapıcısındaki `super(etiket)` satırını silseniz derleyici hangi hatayı verir? Neden?
2. `selam()` metodundan `super.selam()` çağrısını kaldırsanız çıktı nasıl değişir?

Görev 1 — İki Seviyeli Hiyerarşi: `Yayin → Kitap`

Süre	Puan	Zorluk
30 dakika	20 puan	●○○

Amaç

Ortak paydayı bir üst sınıfta toplayıp ondan alt sınıf türetmek; üst sınıfın bıraktığı **uzatma noktasını** ezerek davranışı değiştirmek.

Görev Tanımı

Bir kütüphane için `Yayin` üst sınıfını ve ondan türeyen `Kitap` sınıfını yazın. `Yayin` sınıfı her materyalde bulunan ortak durumu ve ödünç verme mantığını taşısın; `Kitap` sınıfı yalnızca kendine özgü alanları ve **farklı ödünç süresini** eklesin. `Gorev1_YayinDemo` adında bir sürücü programla davranışı gösterin.

Gereksinimler

Üye	Düzey	Beklenen davranış
`demirbasNo`, `baslik`, `yil`	`private final` (Yayin)	Kimlik bilgisi; bir kez atanır
`oduncte`, `oduncAlan`	`protected` (Yayin)	Alt sınıflar kendi kuralında **okuyacak**
`sonrakiSira`	`private static` (Yayin)	Hafta 6'nın sayaç deseni; `D-0001` biçimi
`VARSAYILAN_GUN`	`public static final` = 15	Genel ödünç süresi
`Yayin(String baslik, int yil)`	yapıcı	Boş başlık ve geçersiz yıl `IllegalArgumentException` ile reddedilir
`demirbasNo()`	`public final`	Numara biçimi hiçbir alt sınıfta değiştirilemez
`odunctenGunSayisi()`	`protected`	**Uzatma noktası** — alt sınıflar yalnızca bunu ezer
`oduncVerilebilirMi()`	`public`	Varsayılan: ödünçte değilse evet
`oduncVer(String kime)`, `iadeAl()`	`public`	Durumu değiştirir; ekrana `OK`/`RET` satırı basar
`kunye()`	`public`	Alt sınıflarca ezilecek özet satırı
`Kitap` alanları	`private final`	`yazar`, `sayfaSayisi`, `isbn`
`KITAP_GUN`	`public static final` = 21	Kitaba özgü ödünç süresi
`Kitap` yapıcısı	—	İlk satırı `super(baslik, yil)` olmak **zorunda**
`kalinMi()`	`public`	500 sayfa ve üzeri
`odunctenGunSayisi()`, `kunye()`	`@Override`	İkincisi `super.kunye()` çağrısıyla genişletilir

Beklenen Çıktı

```
▸ TERMINAL — java Gorev1_YayinDemo (birinci bölüm)
=== GOREV 1 : Yayin -> Kitap hiyerarsisi ===

1) Ust sinif nesnesi (genel yayin):
   D-0001 | KTU Of Teknoloji Bulteni (2025)
   odunc suresi -> 15 gun

2) Alt sinif nesnesi (kitap):
   D-0002 | Java How to Program (2018) | Deitel & Deitel, 1024 s., ISBN 978-0134743356
   D-0003 | Effective Java (2018) | Joshua Bloch, 412 s., ISBN 978-0134685991

3) Miras alinan uyeler Kitap'ta YAZILMADI:
   k1.demirbasNo() -> D-0002
   k1.getYil()     -> 2018
   k1.oduncteMi()  -> false
```

```
4) Kitap'a ozgu uyeler:
   k1.kalinMi() -> true (1024 sayfa)
   k2.kalinMi() -> false (412 sayfa)
```

```
▸ TERMINAL — java Gorev1_YayinDemo (ikinci bölüm)
5) Ezilen uzatma noktasinin etkisi:
   Genel yayin oduncu:
  OK : D-0001 -> Elif Demir (15 gun)
   Kitap oduncu (21 gun, cunku odunctenGunSayisi ezildi):
  OK : D-0002 -> Elif Demir (21 gun)
   Ayni kitap ikinci kez:
  RET: D-0002 zaten Elif Demir tarafindan alinmis.

6) Iade ve yeniden odunc:
  OK : D-0002 iade alindi (Elif Demir)
  RET: D-0002 zaten rafta.
  OK : D-0002 -> Murat Kaya (21 gun)
```

DİKKAT

Beklenen çıktıda genel yayın **15 gün**, kitap **21 gün** ile ödünç verilmiştir. Bu farkı üreten kod `oduncVer()` metodu değildir — o metot `Yayin` sınıfında bir kez yazılmıştır. Farkı üreten, `Kitap` sınıfında ezilmiş tek satırlık `odunctenGunSayisi()` metodudur. `oduncVer()` metodunu `Kitap` sınıfında yeniden yazan öğrenci bu görevden puan kaybeder.

İPUCU

Demirbaş numarasını `String.format("D-%04d", sonrakiSira)` ile üretebilirsiniz. Sekizinci bölümdeki toplu listeleme için `Yayin[] raf = { y, k1, k2 };` dizisi yeterlidir; dizinin türü `Yayin` olduğu hâlde içinde `Kitap` nesneleri tutulabildiğine dikkat edin.

SIK YAPILAN HATA

`oduncVer()` metodunun içinde `odunctenGunSayisi()` çağrısını `VARSAYILAN_GUN` sabitiyle **değiştirmeyin**. Sabiti doğrudan yazarsanız alt sınıfların ezmesi hiçbir etki üretmez ve görevin amacı kaybolur.

Kontrol Soruları

1. `oduncte` alanını `private` yapsaydınız hangi görevlerde sorun çıkardı? `protected` yapmanın bedeli nedir?
2. `demirbasNo()` metodunu `final` yapmanın gerekçesi nedir? Bu metot `final` olmasaydı hangi kötü senaryo mümkün olurdu?
3. `Kitap` sınıfında `getYil()` metodunu yazmadınız, ama `k1.getYil()` çalışıyor. Bu nasıl mümkün oluyor?

Görev 2 — Üç Seviyeli Hiyerarşi ve `super` Zinciri

Süre	Puan	Zorluk
25 dakika	20 puan	●●○

Amaç

Üst sınıfın kuralını **kısıtlayarak** genişletmek; üç seviyeli bir hiyerarşide her seviyenin kendi kuralını `super` çağrısıyla birbirine eklemesini sağlamak.

Görev Tanımı

`Yayin` sınıfından türeyen `Dergi` ve `Dergi`den türeyen `AkademikDergi` sınıflarını yazın. Dergilerin **güncel sayısı** ödünç verilmez; akademik dergiler yalnızca akademik personele verilir. İki kural da üst sınıfın mevcut denetimlerini **kaybetmeden** eklenmelidir. `Gorev2_ZincirDemo` adında bir sürücü programla üç seviyeyi birlikte gösterin.

Gereksinimler

Sınıf	Eklenen üye	Ezilen üye ve kural
`Dergi`	`sayiNo`, `periyot`, `protected guncelSayi`, `arsivleyeAl()`	`oduncVerilebilirMi()` — `super.oduncVerilebilirMi() && !guncelSayi`
`Dergi`	`DERGI_GUN` = 7	`odunctenGunSayisi()` ve `kunye()`
`Dergi`	—	`oduncVer()` — güncel sayıysa reddet, değilse `super.oduncVer(kime)`
`AkademikDergi`	`indeks`, `AKADEMIK_GUN` = 3	`odunctenGunSayisi()` ve `kunye()`
`AkademikDergi`	—	`oduncVer()` — ad `"Dr."` veya `"Prof."` ile başlamıyorsa reddet, başlıyorsa `super.oduncVer(kime)`

Beklenen Çıktı

```
▸ TERMINAL — java Gorev2_ZincirDemo (birinci bölüm)
=== GOREV 2 : Yayin -> Dergi -> AkademikDergi ===

1) Dergi (guncel sayi) olusturuluyor:
   D-0001 | Bilim ve Teknik (2026) | sayi 703, aylik, GUNCEL
   oduncVerilebilirMi() -> false
  RET: D-0001 guncel sayi, yalnizca icerde okunur.

2) Arsive alinca kural degisiyor:
  D-0001 arsive alindi, artik odunc verilebilir.
```

```
   oduncVerilebilirMi() -> true
  OK : D-0001 -> Elif Demir (7 gun)
   (7 gun: Dergi.DERGI_GUN)

3) Ust sinif kontrolu KAYBOLMADI:
  RET: D-0001 zaten Elif Demir tarafindan alinmis.
```

```
▸ TERMINAL — java Gorev2_ZincirDemo (ikinci bölüm)
4) Uc seviyeli hiyerarsi : AkademikDergi
   D-0002 | Journal of Software Engineering (2025) | sayi 12, uc aylik, arsiv | indeks: SCI-E

5) Uc seviyenin kurallari SIRAYLA isliyor:
   (a) akademik olmayan kisi:
  RET: D-0002 yalnizca akademik personele verilir (Elif Demir)
   (b) akademik personel:
  OK : D-0002 -> Dr. Ayse Yildiz (3 gun)
   (c) ayni anda ikinci istek:
  RET: D-0002 zaten Dr. Ayse Yildiz tarafindan alinmis.

6) Guncel akademik dergi : iki kural birlikte
   D-0003 | IEEE Software (2026) | sayi 4, uc aylik, GUNCEL | indeks: SCI
  RET: D-0003 guncel sayi, yalnizca icerde okunur.
  D-0003 arsive alindi, artik odunc verilebilir.
  OK : D-0003 -> Dr. Ayse Yildiz (3 gun)
```

```
▸ TERMINAL — java Gorev2_ZincirDemo (üçüncü bölüm)
7) Miras zinciri:
   AkademikDergi -> Dergi -> Yayin -> Object

8) IS-A zinciri:
   a1 instanceof AkademikDergi -> true
   a1 instanceof Dergi         -> true
   a1 instanceof Yayin         -> true
   d1 instanceof AkademikDergi -> false

9) Odunc sureleri (ezilen uzatma noktasi):
   Yayin            15 gun
   Kitap            21 gun
   Dergi            7 gun
   AkademikDergi    3 gun
```

DİKKAT

Çıktının üçüncü bölümünde `AkademikDergi` için üç kural **sırayla** işlemektedir: (a) akademik personel mi? (`AkademikDergi`), (b) güncel sayı mı? (`Dergi`), (c) zaten ödünçte mi? (`Yayin`). Üç kural üç ayrı sınıfta yazılmıştır ve hiçbiri diğerini tekrarlamaz. `super.oduncVer(kime)` çağrısını atlayan bir öğrencide (c) kuralı sessizce kaybolur; "zaten ödünçte" satırı çıktıda görünmez.

İPUCU

Bir metnin belirli bir önekle başlayıp başlamadığını `kime.startsWith("Dr.")` ile sınayabilirsiniz. Miras zincirini

yazdırmak için `getClass()` ile başlayıp `getSuperclass()` çağrısını `null` dönene kadar yineleyen bir `while` döngüsü kurun.

Kontrol Soruları

1. `Dergi.oduncVerilebilirMi()` metodunda `super.oduncVerilebilirMi()` çağrısını silip yalnızca `return !guncelSayi;` yazsanız hangi hata ortaya çıkar? Çıktıda hangi satır kaybolur?
2. `AkademikDergi` sınıfında `guncelSayi` alanına doğrudan erişebiliyorsunuz. Bu hangi erişim düzeyi sayesinde mümkün oluyor? Alan `private` olsaydı ne yapmanız gerekirdi?
3. Üç sınıfın `odunctenGunSayisi()` metotları birbirini çağırmıyor. Bu neden doğru bir tercih? Hangi durumda `super.odunctenGunSayisi()` çağırmak anlamlı olurdu?

Görev 3 — Ortak Davranışı Üst Sınıfa Taşıma

Süre	Puan	Zorluk
25 dakika	20 puan	●●○

Amaç

Var olan bir tasarımdaki kod tekrarını, ortak paydayı bir üst sınıfa taşıyarak (pull up refactoring) gidermek ve kazancı ölçmek.

Başlangıç Durumu

Aşağıdaki iki sınıf size verilmiştir. Birbirinden bağımsız yazılmış olmalarına karşın işaretli satırlar birebir aynıdır.

baslangic_durumu.java

```java
// BASLANGIC DURUMU - Gorev 3'te bunu duzelteceksiniz.
// Iki sinif birbirinden bagimsiz yazilmis; isaretli satirlar KOPYA.
class AkademikPersonelV0{
  private final String ad;            // KOPYA
  private final String sicil;         // KOPYA
  private double brut;                 // KOPYA
  private final String unvan;
  private int haftalikDersSaati;

  double netUcret(){           // KOPYA (govde farkli)
    return brut * 0.85 + dersUcreti();
  }
  void zamAl(double yuzde){        // KOPYA
    if(yuzde <= 0 || yuzde > 100){return;}
```

```
        brut *= (1 + yuzde / 100.0);
    }
    String kunye(){return sicil + "" + ad; }  // KOPYA
    private double dersUcreti(){...}
}

class IdariPersonelV0{
    private final String ad;              // KOPYA
    private final String sicil;           // KOPYA
    private double brut;                  // KOPYA
    private final int kadroDerecesi;
    private int fazlaMesaiSaati;

    double netUcret(){           // KOPYA (govde farkli)
        return brut * 0.82 + mesaiUcreti();
    }
    void zamAl(double yuzde){         // KOPYA
        if(yuzde <= 0 || yuzde > 100){return;}
        brut *= (1 + yuzde / 100.0);
    }
    String kunye(){return sicil + "" + ad; }  // KOPYA
    private double mesaiUcreti(){...}
}
```

Kod 7.1 — Görev 3'ün başlangıç durumu (düzeltilecek)

Görev Tanımı

`Gorev3_OrtakDavranis` adında bir dosya oluşturun. İçinde ortak paydayı taşıyan bir `Personel` üst sınıfı ile ondan türeyen `AkademikPersonel` ve `IdariPersonel` sınıfları bulunsun. `main` metodu üç personel nesnesini bir `Personel[]` dizisinde tutarak tek döngüde işlesin.

Gereksinimler

Sınıf	Taşınan / eklenen	Not
`Personel`	`protected ad`, `protected sicil`, `protected brut`	Alt sınıflar `kunye()` içinde okuyacak
`Personel`	`zamAl(double)`, `kunye()`, `netUcret()`	Üçü de **tek** yerde yazılır
`Personel`	`protected kesintiOrani()` → `0.15`	Uzatma noktası
`Personel`	`protected ek()` → `0.0`	İkinci uzatma noktası; varsayılan ek yok
`AkademikPersonel`	`unvan`, `haftalikDersSaati`, `dersSaatiAta(int)`	İlk 10 saat kadroda; fazlası ücretlendirilir
`AkademikPersonel`	`ek()` ve `kunye()` ezilir	`kunye()` içinde `super.kunye()` çağrılır
`IdariPersonel`	`kadroDerecesi`, `fazlaMesaiSaati`, `mesaiEkle(int)`	Kadro derecesi 1–15 aralığında doğrulanır

`IdariPersonel`	`kesintiOrani()` ve `ek()` ezilir	3. derece ve altında oran `0.18`
Her iki alt sınıf	`netUcret()` **yazılmaz**	Yalnızca uzatma noktaları ezilir

Beklenen Çıktı

```
▸ TERMINAL — java Gorev3_OrtakDavranis (birinci bölüm)
=== GOREV 3 : ortak davranisi ust sinifa tasima ===

1) Baslangic durumu:
   P-5001 | Ayse Yildiz      | brut  42000.00 | net  45780.00 | Dr. Ogr. Uyesi, 16 saat/hafta
   P-5002 | Murat Kaya       | brut  28000.00 | net  22960.00 | 2. derece, 0 saat mesai
   P-5003 | Zeynep Ak        | brut  24000.00 | net  20400.00 | 7. derece, 0 saat mesai

2) Ust sinifta TEK yerde tanimli zamAl() herkese isliyor:
   P-5001 | Ayse Yildiz      | brut  47250.00 | net  50242.50 | Dr. Ogr. Uyesi, 16 saat/hafta
   P-5002 | Murat Kaya       | brut  31500.00 | net  25830.00 | 2. derece, 0 saat mesai
   P-5003 | Zeynep Ak        | brut  27000.00 | net  22950.00 | 7. derece, 0 saat mesai

3) Ezilen ek() metodu:
   AkademikPersonel.ek() -> 20160.00
   P-5001 | Ayse Yildiz      | brut  47250.00 | net  60322.50 | Dr. Ogr. Uyesi, 22 saat/hafta
   P-5002 | Murat Kaya       | brut  31500.00 | net  29160.00 | 2. derece, 18 saat mesai
```

```
▸ TERMINAL — java Gorev3_OrtakDavranis (ikinci bölüm)

4) Ezilen kesintiOrani() : netUcret() DEGISTIRILMEDI
   ip1 (2. derece) kesinti orani -> 0.18
   ip2 (7. derece) kesinti orani -> 0.15
   P-5003 | Zeynep Ak        | brut  27000.00 | net  22950.00 | 7. derece, 0 saat mesai

5) Gecersiz girdiler ust sinifta yakalaniyor:
  UYARI: zam yuzdesi 0-100 araliginda olmalidir.
  UYARI: mesai saati pozitif olmalidir.
  UYARI: ders saati 0-40 araliginda olmalidir.

6) Olculebilir kazanim:
   Once : 2 sinif x ~35 satir = ~70 satir, ~45'i kopya
   Sonra: 1 ust (68 satir) + 2 alt (~40 satir), kopya 0
   Zam kurali degisirse tek dosya duzenlenir.
```

İPUCU

Bu görevin ölçüsü satır sayısıdır. Başlangıç durumunda iki sınıfta yaklaşık 45 satır kopyaydı; doğru çözümde kopya satır sayısı **sıfır** olmalıdır. Çözümünüzü teslim etmeden önce iki alt sınıfı yan yana açıp aynı satırı arayın; bulursanız o satır üst sınıfa taşınmalıdır.

DİKKAT

`netUcret()` metodunu alt sınıflarda **yeniden yazmayın**. Doğru çözümde bu metot yalnızca `Personel` sınıfında bulunur ve iki uzatma noktasını (`kesintiOrani()` ve `ek()`) çağırır. Metodu alt sınıflarda tekrar yazan öğrenci, düzeltmeye çalıştığı tekrarı farklı bir yerde yeniden üretmiş olur.

Kontrol Soruları

1. Zam kuralı değişse (örneğin üst sınır %100 yerine %50 olsa) başlangıç durumunda kaç dosya, sizin çözümünüzde kaç dosya düzeltilir?
2. `kesintiOrani()` metodunu `protected` yerine `private` yapsaydınız `IdariPersonel` sınıfı ne yapamazdı?
3. `Personel` sınıfını `abstract` yapmak (Hafta 10) bu tasarımı nasıl iyileştirirdi? Şu anda `new Personel(...)` çağrısı yapılabiliyor; bu anlamlı mı?

Görev 4 — IS-A / HAS-A Kararı

Süre	Puan	Zorluk
20 dakika	20 puan	●●○

Amaç

Bir tasarım kararını cümle sınamasıyla gerekçelendirmek; yanlış kurulmuş bir kalıtımı bileşime dönüştürmek ve düzeltmenin somut kazancını göstermek.

Başlangıç Durumu

Bir önceki dönem yazılmış aşağıdaki sınıf size verilmiştir. Cümleyi yüksek sesle okuyunca sorun ortaya çıkar.

bozuk_tasarim.java

```java
// BOZUK TASARIM - Gorev 4'te bunu duzelteceksiniz.
class BozukOduncKaydi extends Uye {   // "Odunc kaydi BIR uyedir"?
  private final String demirbasNo;
  private int gun;

  BozukOduncKaydi(String uyeNo, String uyeAd, String demirbasNo, int gun){
    super(uyeNo, uyeAd);      // her kayit YENI bir uye uretiyor
    this.demirbasNo = demirbasNo;
    this.gun = gun;
  }
}
```

Kod 7.2 — Görev 4'ün bozuk tasarımı (düzeltilecek)

Görev Tanımı

`Gorev4_IsaHasa` adında bir dosya oluşturun. İçinde bir `Uye` sınıfı, verilen bozuk `BozukOduncKaydi` sınıfı ve sizin yazacağınız **doğru** `OduncKaydi` sınıfı bulunsun. `main` metodu önce bozuk tasarımın üç zararını göstersin, sonra doğru tasarımın aynı senaryoyu nasıl çözdüğünü kanıtlasın.

Gereksinimler

Bölüm	Beklenen
`Uye` sınıfı	`no`, `ad`, `cezaPuani` alanları; `cezaEkle(int)`, `kisitliMi()` (30 puan eşiği), `kunye()`
Bozuk tasarımın gösterimi	Aynı üyenin **iki** kaydı oluşturulup ceza puanlarının **bölündüğü** kanıtlanır
`OduncKaydi` sınıfı	`private final Uye uye` alanı (HAS-A) + `demirbasNo`, `gun`, `kapali`
Yapıcı doğrulaması	`uye == null` ise `IllegalArgumentException`; `uye.kisitliMi()` ise `IllegalStateException`
`kapat(int gecikmeGunu)`	Gecikme varsa `uye.cezaEkle(gecikmeGunu * 2)` — **delegasyon**
Paylaşım kanıtı	İki kaydın `uye()` çağrısı `==` ile karşılaştırılıp `true` olduğu gösterilir
Karar tablosu	Altı senaryo için IS-A / HAS-A ve karşılığı yazdırılır

Beklenen Çıktı

```
▸ TERMINAL — java Gorev4_IsaHasa (bozuk tasarımın zararları)
=== GOREV 4 : IS-A / HAS-A karari ===

1) BOZUK tasarimin sonuclari:
   bozuk.kunye() -> U-100 Elif Demir (ceza 0)
   (a) Odunc kaydinin CEZA PUANI oldu:
       bozuk.cezaPuani() -> 10
   (b) bozuk instanceof Uye -> true  ('kayit bir uyedir' anlamsiz)
   (c) Ayni uyenin IKI kaydi olursa uyenin cezasi
       iki KOPYAYA bolunur; tek gercek yoktur.
       kayit1.cezaPuani() -> 10
       kayit2.cezaPuani() -> 25
       Ayni uye, iki farkli ceza. VERI BOLUNDU.

2) DOGRU tasarim (bilesim):
   D-0001 -> U-100 Elif Demir (ceza 0), 21 gun, ACIK
   D-0002 -> U-100 Elif Demir (ceza 0), 7 gun, ACIK
   Iki kayit AYNI uye nesnesini paylasiyor:
   k1.uye() == k2.uye() -> true
```

```
▸ TERMINAL — java Gorev4_IsaHasa (doğru tasarım)
```

```
3) Ceza tek yerde birikiyor:
  D-0001 gec iade (4 gun) -> Elif Demir ceza aldi
  D-0002 zamaninda iade edildi
   elif.kunye() -> U-100 Elif Demir (ceza 8)
   elif.kisitliMi() -> false

4) Kisit is kuralina donusebiliyor:
   elif.kunye() -> U-100 Elif Demir (ceza 33)
   Beklenen ret: Kisitli uyeye odunc verilemez: Elif Demir
```

SIK YAPILAN HATA

Bozuk tasarımın en yıkıcı sonucu çıktının 10.–12. satırlarındadır: aynı üyenin iki kaydı oluşturulduğunda **iki ayrı ceza puanı** doğar ve hangisinin gerçek olduğu belirsizleşir. Kalıtım burada yalnızca "yanlış" değil, **veri bütünlüğünü bozan** bir karardır.

İPUCU

İki referansın aynı nesneyi gösterip göstermediğini `==` ile sınayın; `equals()` kullanmayın (Hafta 9'a kadar `equals()` varsayılan olarak `==` ile aynı sonucu verir, ama niyetiniz kimlik karşılaştırmasıdır). Karar tablosunu hizalamak için `System.out.printf("  %-14s %-9s %-6s %s%n", ...)` kullanabilirsiniz.

Kontrol Soruları

1. "Ödünç kaydı bir üyedir" cümlesi neden anlamsızdır? Cümleyi anlamlı hâle getiren doğru ifade nedir?
2. Bozuk tasarımda `BozukOduncKaydi` nesnesinin `cezaEkle(10)` metodunu çağırabildiniz. Bu, kalıtımın hangi genel sorununun örneğidir?
3. Doğru tasarımda `OduncKaydi` sınıfı `Uye` sınıfının bütün metotlarını dışarıya açmıyor. Bu bir eksiklik mi, yoksa bileşimin sağladığı bir avantaj mı? Gerekçelendiriniz.

Görev 5 (Bonus) — Kalıtımdan Bileşime: Katalog

Süre	Puan	Zorluk
20 dakika	20 puan (bonus)	●●●

Amaç

Görev 1 ve 2'de kurulan hiyerarşinin tamamını tek bir kap sınıf üzerinden çalıştırmak; bu kap sınıfın neden alt sınıf değil **sahip** olması gerektiğini göstermek.

Görev Tanımı

`Gorev5_Kutuphane` adında bir dosya oluşturun. İçinde bilinçli olarak yanlış yazılmış bir `BozukKatalog extends Yayin` sınıfı ile doğru tasarlanmış bir `Katalog` sınıfı bulunsun. `Katalog` sınıfı `Yayin[]` dizisi tutsun ve hiyerarşinin tamamıyla **üst tür referansı** üzerinden çalışsın.

Gereksinimler

Üye	Beklenen davranış
`BozukKatalog extends Yayin`	Yanlış tasarımın derlendiğini ve `bk.oduncVer(...)` çağrısının **çalıştığını** göstermek için bilinçli olarak yazılır
`Katalog(String ad, int kapasite)`	Kapasite pozitif olmalı; `Yayin[] raf` alanı oluşturulur
`ekle(Yayin y)`	`null` ve dolu raf durumlarında uyarı basıp `false` döner
`oduncVerTumu(String kime)`	Raftaki her yayına `oduncVer(kime)` çağırır; başarılı sayısını döner
`oduncSayisi()`	Ödünçte olan yayın sayısını sayar
`ara(String demirbasNo)`	Bulamazsa `null` döner
`listele()`	`getClass().getSimpleName()` ile her yayının **gerçek türünü** ve `kunye()` çıktısını yazdırır
`main`	İki kitap, bir dergi ve bir akademik dergi ekleyip iki farklı kişiyle `oduncVerTumu` çağırır

Beklenen Çıktı

```
▸ TERMINAL — java Gorev5_Kutuphane (bozuk tasarım ve katalog)
=== GOREV 5 (BONUS) : kalitimdan bilesime ===

1) BOZUK tasarim neden anlamsiz:
   bk.kunye()      -> D-0001 | Merkez Katalogu (2026)
   bk.oduncteMi()  -> false   (katalog odunc verilebilir mi?)
   Katalog bir raftir, bir yayin degildir.
  OK : D-0001 -> Elif Demir (15 gun)
   Yukaridaki satir DERLENDI ve CALISTI: tasarim hatasi
   derleyici tarafindan yakalanamaz.

2) DOGRU tasarim: Katalog HAS-A Yayin[]
  UYARI: bos yayin eklenemez.
  --- Of Teknoloji Fakultesi Kitapligi (4/5) ---
  Kitap          D-0003 | Core Java Volume I (2022) | Cay Horstmann, 928 s., ISBN 978-0137673629
  Kitap          D-0004 | Java: The Complete Reference (2021) | Herbert Schildt, 1248 s., ISBN
978-1260463415
  Dergi          D-0005 | Bilim ve Teknik (2026) | sayi 703, aylik, arsiv
  AkademikDergi  D-0006 | ACM TOSEM (2025) | sayi 34, uc aylik, arsiv | indeks: SCI-E
```

```
▸ TERMINAL — java Gorev5_Kutuphane (üst tür üzerinden toplu işlem)
3) Ust tur uzerinden toplu islem (Hafta 09 onizlemesi):
   oduncVerTumu("Elif Demir"):
  OK : D-0003 -> Elif Demir (21 gun)
  OK : D-0004 -> Elif Demir (21 gun)
```

```
  OK : D-0005 -> Elif Demir (7 gun)
  RET: D-0006 yalnizca akademik personele verilir (Elif Demir)
   basarili: 3 / 4
   Katalog, AkademikDergi kuralini BILMEDEN dogru
   sonucu uretti; her nesne kendi kuralini isletti.

4) Akademik personel ile yeniden:
   oduncVerTumu("Dr. Ayse Yildiz"):
  RET: D-0003 zaten Elif Demir tarafindan alinmis.
  RET: D-0004 zaten Elif Demir tarafindan alinmis.
  RET: D-0005 zaten Elif Demir tarafindan alinmis.
  OK : D-0006 -> Dr. Ayse Yildiz (3 gun)
   basarili: 1 / 4
   oduncte olan yayin sayisi -> 4
```

> **BİLGİ**
>
> Bu görev Hafta 9'un (çok biçimlilik) kapısını açar. `Katalog` sınıfı elindeki nesnenin `Kitap`, `Dergi` yoksa `AkademikDergi` olduğunu **bilmemektedir**; yine de her nesne kendi ödünç kuralını işletmiştir. Çıktıda 4 yayından 3'ünün verilip birinin reddedilmesi bunun kanıtıdır.

> **DİKKAT**
>
> `oduncVerTumu()` metodunun içinde `instanceof` kullanarak tür ayırt etmeyin. Doğru çözümde yalnızca `raf[i].oduncVer(kime)` çağrısı bulunur; hangi kuralın işleyeceğini nesnenin kendisi belirler. `instanceof` zinciri yazan öğrenci bonus puanın yarısını kaybeder.

Kontrol Soruları

1. `BozukKatalog` sınıfı derlendi ve `oduncVer(...)` çağrısı çalıştı. Derleyicinin bu tasarım hatasını yakalamaması ne anlama gelir?
2. `Katalog` sınıfı `Yayin` sınıfından türemediği hâlde `Yayin[]` dizisi tutabiliyor. Bu iki durum arasındaki ilişkiyi IS-A / HAS-A terimleriyle açıklayınız.
3. `oduncVerTumu()` metodu içinde hangi sınıfın `oduncVer()` metodunun çalışacağı **ne zaman** belirlenir: derleme zamanında mı, çalışma zamanında mı? Yanıtınızı gerekçelendiriniz.

Değerlendirme ve Teslim

Puanlama Ölçütleri

Görev	Ölçüt	Puan
Görev 1	`Yayin` üst sınıfı doğru kurulmuş (ortak payda, `protected` gerekçeli, `final` numara)	8
Görev 1	`Kitap` yalnızca uzatma noktasını ezmiş; `oduncVer()` yeniden yazılmamış	8
Görev 1	Beklenen çıktı birebir üretilmiş	4
Görev 2	`Dergi` ve `AkademikDergi` kuralları `super` çağrısını **koruyarak** eklemiş	10
Görev 2	Üç seviyede `@Override` ve doğru ödünç süreleri	6
Görev 2	Miras zinciri ve `instanceof` sınamaları doğru	4
Görev 3	Ortak payda üst sınıfa taşınmış; kopya satır **sıfır**	10
Görev 3	`netUcret()` yalnızca üst sınıfta; iki uzatma noktası ezilmiş	6
Görev 3	Doğrulamalar üst sınıfta, tek noktada	4
Görev 4	Bozuk tasarımın üç zararı çalışan çıktıyla gösterilmiş	8
Görev 4	`OduncKaydi` bileşimle yazılmış; delegasyon uygulanmış	8
Görev 4	Karar tablosu ve gerekçeler doğru	4
Görev 5	`Katalog` bileşimle yazılmış; `instanceof` yok	12 (bonus)
Görev 5	Üst tür üzerinden toplu işlem doğru çalışıyor	8 (bonus)
Tümü	Kod düzeni, adlandırma, açıklama yorumları	Diğer ölçütlere yayılmıştır

Tablo 7.13 — Görev bazında puan dağılımı (toplam 80 + 20 bonus)

Teslim Öncesi Kontrol Listesi

- [ ] Bütün dosyalar hatasız derleniyor: `javac -d bin src/*.java`
- [ ] Ezdiğim **her** metodun üstünde `@Override` var.

Üst sınıf kuralını genişlettiğim yerlerde `super.metot()` çağrısı var; olmayan yerlerde neden olmadığını yorumla yazdım.

Alt sınıflarda üst sınıftaki bir alan adını **tekrar kullanmadım**.

`protected` yaptığım her üye için gerekçemi yorum satırına yazdım.

`netUcret()` ve `oduncVer()` gibi ortak metotları alt sınıflarda **yeniden yazmadım**.

Görev 5'te `instanceof` ile tür ayırt etmedim.

Kaynak dosyalarda Türkçe karakter yok (kod ve yorumlar ASCII).

Her `public` sınıf kendi adıyla ayrı bir dosyada.

Zip dosyası adı `Hafta07_<Numara>_<AdSoyad>.zip` biçiminde.

Sık Karşılaşılan Hatalar

Belirti	Olası neden	Çözüm
`Implicit super constructor Yayin() is undefined`	Alt sınıf yapıcısında `super(...)` yazılmamış	İlk satıra `super(baslik, yil)` ekleyin
`Constructor call must be the first statement`	`super(...)` başka bir deyimden sonra yazılmış	Çağrıyı yapıcının en başına taşıyın
Kitap da 15 gün ile ödünç veriliyor	`oduncVer()` içinde sabit doğrudan yazılmış	`odunctenGunSayisi()` çağrısını kullanın
“Zaten ödünçte” satırı hiç görünmüyor	`super.oduncVer(kime)` çağrısı atlanmış	Üst sınıf çağrısını geri ekleyin
`The method f(String) ... must override`	İmza üst sınıftakiyle uyuşmuyor	Parametre türünü üst sınıftakiyle eşitleyin
Aynı üyenin cezası iki farklı değer gösteriyor	Kalıtım kullanılmış; her kayıt yeni bir üye üretiyor	Bileşime geçin: `Uye` bir alan olsun
`guncelSayi` alanına erişilemiyor	Alan `private` bırakılmış	`protected` yapın veya `protected` getter ekleyin
Katalog ödünç verilebiliyor	`Katalog extends Yayin` yazılmış	`Yayin[] raf` alanına geçin

Tablo 7.14 — Laboratuvarda en sık görülen sorunlar ve çözümleri

> **BİLGİ**
>
> Teslim ettiğiniz kodun **çalışıyor** olması yeterli değildir; tasarım kararlarınızı gerekçelendirmeniz beklenir. Bu yüzden her `extends` bildiriminin üstüne, kurduğunuz IS-A cümlesini bir yorum satırı olarak yazın. Örnek: `// "Kitap bir Yayindir" -> IS-A -> extends`